

7차 회의록

WakeBuddy

AI에게 바로 설계를 맡기는 방식으로 진행하여 어플리케이션 제작

오늘은 전통적인 개발 절차인 요구사항 분석, 설계, 구현, 테스트, 품질관리 단계를 순차적으로 수행하는 방식이 아니라,

AI에게 어플리케이션 설계를 직접 맡기고, 이를 기반으로 어플리케이션 제작하는 것을 목표로 한다.

[어플리케이션 소개]

공유 알람 어플리케이션은 사용자가 로그인 후 본인의 알람을 조회, 생성, 수정, 삭제할 수 있을 뿐만 아니라, 친구로 등록된 사용자의 알람도 동일하게 관리할 수 있는 서비스이다.

사용자는 친구 추가 요청을 보낼 수 있으며, 상대방이 해당 요청을 수락한 경우에만 친구 관계가 성립된다. 친구 관계가 성립된 이후에는 서로의 알람을 확인하거나 새 알람을 생성하고, 기존 알람을 수정 또는 삭제할 수 있다.

알람 기능은 기본 알람 어플리케이션과 유사하게 동작한다. 사용자는 원하는 시간과 요일을 설정할 수 있으며, 특정 요일마다 반복되는 알람도 등록할 수 있다. 또한 등록된 알람은 필요에 따라 활성화하거나 비활성화할 수 있어, 알람을 삭제하지 않고도 일시적으로 꺼두거나 다시 켤 수 있다.

Android와 iOS에서 모두 실행 가능한 크로스플랫폼 어플리케이션으로 제작하고 싶으며, 어플리케이션 개발에 필요한 전체 코드를 작성해 줘.
간단히 ^

AI 답변

요구사항을 실제 개발 가능한 범위로 정리해서, Android/iOS 공통으로 돌릴 수 있는 React Native 기반 MVP 구조와 핵심 전체 코드를 한 번에 잡아줄게요. 알람의 OS 백그라운드 동작은 플랫폼 제약이 있어서 "앱 내부 알람 + 로컬 알림" 기준으로 설계하겠습니다.

15s 동안 생각함

아래는 **Android/iOS 공통 실행 가능한 React Native + Expo + Firebase 기반 MVP 전체 코드**입니다.

구현 범위는 다음과 같습니다.

- 로그인 / 회원가입
- 내 알람 조회, 생성, 수정, 삭제
- 친구 검색
- 친구 요청 보내기
- 친구 요청 수락
- 친구 목록 조회
- 친구의 알람 조회, 생성, 수정, 삭제
- 알람 활성화 / 비활성화
- 요일 반복 알람
- Expo 로컬 알림 기반 알람 등록

Expo Notifications는 로컬 알림과 푸시 알림을 모두 지원하고, 특정 시간 또는 반복 조건으로 알림을 예약할 수 있습니다.

데이터 권한 관리는 Firebase Authentication과 Firestore Security Rules 조합으로 처리하는 구조입니다. Firebase 문서에서도 Security Rules가 인증 여부와 데이터 접근 조건을 검사하는 방식이라고 설명합니다.

화면 이동은 React Navigation의 native stack 구조를 사용합니다. React Navigation은 React Native 앱에서 iOS/Android 화면 전환을 처리하는 대표적인 네비게이션 라이브러리입니다.

1. 프로젝트 생성

```
npx create-expo-app shared-alarm-app--template blank-typescript
cd shared-alarm-app
```

2. 패키지 설치

```
npm install firebase
npm install @react-navigation/native @react-navigation/native-stack
npm install react-native-screens react-native-safe-area-context
npx expo install expo-notifications
```

3. 폴더 구조

```
shared-alarm-app
├─ App.tsx
├─ firebaseConfig.ts
├─ firestore.rules
├─ src
│  ├─ types.ts
│  ├─ navigation
│  │  └─ RootNavigator.tsx
│  ├─ contexts
│  │  └─ AuthContext.tsx
│  ├─ services
│  │  ├─ alarmService.ts
│  │  ├─ friendService.ts
│  │  └─ notificationService.ts
│  ├─ screens
│  │  ├─ LoginScreen.tsx
│  │  ├─ RegisterScreen.tsx
│  │  ├─ HomeScreen.tsx
│  │  ├─ AlarmFormScreen.tsx
│  │  ├─ FriendsScreen.tsx
│  │  └─ FriendAlarmsScreen.tsx
│  └─ components
│      └─ AlarmItem.tsx
```

4. 코드

firebaseConfig.ts

Firebase 콘솔에서 본인 프로젝트 설정값으로 바꾸면 됩니다.

```

import {initializeApp }from"firebase/app";
import {getAuth }from"firebase/auth";
import {getFirestore }from"firebase/firestore";

constfirebaseConfig= {
  apiKey:"YOUR_API_KEY",
  authDomain:"YOUR_PROJECT.firebaseio.com",
  projectId:"YOUR_PROJECT_ID",
  storageBucket:"YOUR_PROJECT.appspot.com",
  messagingSenderId:"YOUR_SENDER_ID",
  appId:"YOUR_APP_ID",
};

constapp=initializeApp(firebaseConfig);

exportconstauth=getAuth(app);
exportconstdb=getFirestore(app);

```

src/types.ts

```

exporttypeRootStackParamList= {
  Login:undefined;
  Register:undefined;
  Home:undefined;
  AlarmForm: {
    ownerId:string;
    alarmId?:string;
  };
  Friends:undefined;
  FriendAlarms: {
    friendId:string;
    friendEmail:string;
  };
};

exporttypeUserProfile= {
  uid:string;
  email:string;
  createdAt:number;
};

exporttypeAlarm= {
  id:string;
  ownerId:string;
  title:string;
  hour:number;
  minute:number;
  repeatDays:number[];
  enabled:boolean;
  createdBy:string;
};

```

```

    createdAt:number;
    updatedAt:number;
    notificationIds?:string[];
  };

  exporttypeFriendRequest= {
    id:string;
    fromUid:string;
    fromEmail:string;
    toUid:string;
    toEmail:string;
    status:"pending"|"accepted"|"rejected";
    createdAt:number;
  };

  exporttypeFriend= {
    uid:string;
    email:string;
  };

```

src/contexts/AuthContext.tsx

```

import React, { createContext, useContext, useEffect, useState } from "react";
import {
  User,
  createUserWithEmailAndPassword,
  signInWithEmailAndPassword,
  signOut,
  onAuthStateChanged,
} from "firebase/auth";
import { doc, setDoc, getDoc } from "firebase/firestore";
import { auth, db } from "../../firebaseConfig";

type AuthContextValue = {
  user: User | null;
  loading: boolean;
  login: (email: string, password: string) => Promise<void>;
  register: (email: string, password: string) => Promise<void>;
  logout: () => Promise<void>;
};

const AuthContext = createContext<AuthContextValue | null>(null);

export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const unsubscribe = onAuthStateChanged(auth, (currentUser) => {
      setUser(currentUser);
      setLoading(false);
    });
  });

```

```

return unsub;
}, []);

const register = async (email: string, password: string) => {
  const result = await createUserWithEmailAndPassword(auth, email, password);
  const uid = result.user.uid;

  const userRef = doc(db, "users", uid);
  const snap = await getDoc(userRef);

  if (!snap.exists()) {
    await setDoc(userRef, {
      uid,
      email,
      createdAt: Date.now(),
    });
  }
};

const login = async (email: string, password: string) => {
  await signInWithEmailAndPassword(auth, email, password);
};

const logout = async () => {
  await signOut(auth);
};

return (
  <AuthContext.Provider value={{ user, loading, login, register, logout }}>
    {children}
  </AuthContext.Provider>
);
}

export function useAuth() {
  const value = useContext(AuthContext);

  if (!value) {
    throw new Error("useAuth must be used inside AuthProvider");
  }

  return value;
}

```

src/services/notificationService.ts

```

import * as Notifications from "expo-notifications";
import { Platform } from "react-native";
import { Alarm } from "../types";

Notifications.setNotificationHandler({

```

```

    handleNotification:async () => ({
        shouldShowBanner:true,
        shouldShowList:true,
        shouldPlaySound:true,
        shouldSetBadge:false,
    }),
});

export async function requestNotificationPermission() {
    const { status } = await Notifications.requestPermissionsAsync();

    if (Platform.OS === "android") {
        await Notifications.setNotificationChannelAsync("alarm", {
            name: "Alarm",
            importance: Notifications.AndroidImportance.MAX,
            sound: "default",
            vibrationPattern: [0, 500, 300, 500],
        });
    }

    return status === "granted";
}

export async function cancelAlarmNotifications(notificationIds?: string[]) {
    if (!notificationIds || notificationIds.length === 0) return;

    for (const id of notificationIds) {
        await Notifications.cancelScheduledNotificationAsync(id);
    }
}

export async function scheduleAlarmNotifications(alarm: Alarm) {
    if (!alarm.enabled) return [];

    const ids: string[] = [];

    if (alarm.repeatDays.length === 0) {
        const now = new Date();
        const target = new Date();

        target.setHours(alarm.hour);
        target.setMinutes(alarm.minute);
        target.setSeconds(0);
        target.setMilliseconds(0);

        if (target <= now) {
            target.setDate(target.getDate() + 1);
        }

        const id = await Notifications.scheduleNotificationAsync({
            content: {
                title: "알람",
                body: alarm.title,
                sound: "default",
            },
            trigger: {

```

```

        type:Notifications.SchedulableTriggerInputTypes.DATE,
        date:target,
        channelId:"alarm",
      },
    });

    ids.push(id);
    returnids;
  }

  for (constdayofalarm.repeatDays) {
    constid=awaitNotifications.scheduleNotificationAsync({
      content: {
        title:"반복 알람",
        body:alarm.title,
        sound:"default",
      },
      trigger: {
        type:Notifications.SchedulableTriggerInputTypes.WEEKLY,
        weekday:day,
        hour:alarm.hour,
        minute:alarm.minute,
        channelId:"alarm",
      },
    });

    ids.push(id);
  }

  returnids;
}

```

요일 값은 Expo 기준으로 1 = 일요일, 2 = 월요일, ..., 7 = 토요일 로 사용합니다.

src/services/alarmService.ts

```

import {
  addDoc,
  collection,
  deleteDoc,
  doc,
  getDoc,
  onSnapshot,
  orderBy,
  query,
  updateDoc,
}from"firebase/firestore";
import {db }from"../firebaseConfig";
import {Alarm }from"../types";
import {
  cancelAlarmNotifications,

```

```

scheduleAlarmNotifications,
}from"./notificationService";

exportfunctionsubscribeAlarms(
  ownerId:string,
  callback: (alarms:Alarm[]) =>void
) {
  constq=query(
    collection(db,"users",ownerId,"alarms"),
    orderBy("hour","asc"),
    orderBy("minute","asc")
  );

  returnonSnapshot(q, (snapshot) => {
    constalarms:Alarm[]=snapshot.docs.map((d) => ({
      id:d.id,
      ...(d.data()asOmit<Alarm,"id">),
    }));

    callback(alarms);
  });
}

exportasyncfunctiongetAlarm(ownerId:string,alarmId:string) {
  constref=doc(db,"users",ownerId,"alarms",alarmId);
  constsnap=awaitgetDoc(ref);

  if (!snap.exists())returnnull;

  return {
    id:snap.id,
    ...(snap.data()asOmit<Alarm,"id">),
  }asAlarm;
}

exportasyncfunctioncreateAlarm(
  ownerId:string,
  currentUserId:string,
  data: {
    title:string;
    hour:number;
    minute:number;
    repeatDays:number[];
    enabled:boolean;
  }
) {
  consttempAlarm:Alarm= {
    id:"",
    ownerId,
    title:data.title,
    hour:data.hour,
    minute:data.minute,
    repeatDays:data.repeatDays,
    enabled:data.enabled,
    createdBy:currentUserId,
    createdAt>Date.now(),
  }
}

```

```

        updatedAt: Date.now(),
        notificationIds: [],
    };

    const notificationIds =
    ownerId === currentUserId ? await scheduleAlarmNotifications(tempAlarm): [];

    await addDoc(collection(db, "users", ownerId, "alarms"), {
        ...tempAlarm,
        notificationIds,
    });
}

export async function updateAlarm(
    ownerId: string,
    currentUserId: string,
    alarmId: string,
    data: {
        title: string;
        hour: number;
        minute: number;
        repeatDays: number[];
        enabled: boolean;
    }
) {
    const oldAlarm = await getAlarm(ownerId, alarmId);

    if (oldAlarm && ownerId === currentUserId) {
        await cancelAlarmNotifications(oldAlarm.notificationIds);
    }

    const updatedAlarm: Alarm = {
        id: alarmId,
        ownerId,
        title: data.title,
        hour: data.hour,
        minute: data.minute,
        repeatDays: data.repeatDays,
        enabled: data.enabled,
        createdBy: oldAlarm?.createdBy ?? currentUserId,
        createdAt: oldAlarm?.createdAt ?? Date.now(),
        updatedAt: Date.now(),
        notificationIds: [],
    };

    const notificationIds =
    ownerId === currentUserId
    ? await scheduleAlarmNotifications(updatedAlarm)
    : oldAlarm?.notificationIds ?? [];

    await updateDoc(doc(db, "users", ownerId, "alarms", alarmId), {
        title: data.title,
        hour: data.hour,
        minute: data.minute,
        repeatDays: data.repeatDays,
        enabled: data.enabled,
    });
}

```

```

        updatedAt:Date.now(),
        notificationIds,
    });
}

export async function toggleAlarm(
  ownerId:string,
  currentUserId:string,
  alarm:Alarm
) {
  await updateAlarm(ownerId,currentUserId,alarm.id, {
    title:alarm.title,
    hour:alarm.hour,
    minute:alarm.minute,
    repeatDays:alarm.repeatDays,
    enabled:!alarm.enabled,
  });
}

export async function deleteAlarm(
  ownerId:string,
  currentUserId:string,
  alarm:Alarm
) {
  if (ownerId===currentUserId) {
    await cancelAlarmNotifications(alarm.notificationIds);
  }

  await deleteDoc(doc(db,"users",ownerId,"alarms",alarm.id));
}

```

src/services/friendService.ts

```

import {
  addDoc,
  collection,
  doc,
  getDocs,
  onSnapshot,
  query,
  setDoc,
  updateDoc,
  where,
} from "firebase/firestore";
import {db } from "../../firebaseConfig";
import {Friend,FriendRequest } from "../../types";

export async function findUserByEmail(email:string) {
  const q=query(collection(db,"users"),where("email","==",email.trim()));
  const snap=await getDocs(q);

  if (snap.empty) return null;

```

```

const userDoc = snap.docs[0];

return {
  uid: userDoc.id,
  email: userDoc.data().email,
};
}

export async function sendFriendRequest(
  fromUid: string,
  fromEmail: string,
  toEmail: string
) {
  const target = await findUserByEmail(toEmail);

  if (!target) {
    throw new Error("해당 이메일의 사용자를 찾을 수 없습니다.");
  }

  if (target.uid === fromUid) {
    throw new Error("본인에게 친구 요청을 보낼 수 없습니다.");
  }

  const duplicated = query(
    collection(db, "friendRequests"),
    where("fromUid", "=", fromUid),
    where("toUid", "=", target.uid),
    where("status", "=", "pending")
  );

  const duplicatedSnap = await getDocs(duplicated);

  if (!duplicatedSnap.empty) {
    throw new Error("이미 보낸 친구 요청입니다.");
  }

  await addDoc(collection(db, "friendRequests"), {
    fromUid,
    fromEmail,
    toUid: target.uid,
    toEmail: target.email,
    status: "pending",
    createdAt: Date.now(),
  });
}

export function subscribeIncomingRequests(
  uid: string,
  callback: (requests: FriendRequest[]) => void
) {
  const q = query(
    collection(db, "friendRequests"),
    where("toUid", "=", uid),
    where("status", "=", "pending")
  );
}

```

```

returnonSnapshot(q, (snapshot) => {
constrequests=snapshot.docs.map((d) => ({
    id:d.id,
    ...(d.data()asOmit<FriendRequest,"id">),
}));

callback(requests);
});
}

exportasyncfunctionacceptFriendRequest(request:FriendRequest) {
awaitupdateDoc(doc(db,"friendRequests",request.id), {
    status:"accepted",
});

awaitsetDoc(doc(db,"users",request.fromUid,"friends",request.toUid), {
    uid:request.toUid,
    email:request.toEmail,
    createdAt:Date.now(),
});

awaitsetDoc(doc(db,"users",request.toUid,"friends",request.fromUid), {
    uid:request.fromUid,
    email:request.fromEmail,
    createdAt:Date.now(),
});
}

exportfunctionsubscribeFriends(
uid:string,
callback: (friends:Friend[]) =>void
) {
returnonSnapshot(collection(db,"users",uid,"friends"), (snapshot) => {
constfriends=snapshot.docs.map((d) => ({
    uid:d.id,
    email:d.data().email,
}));

callback(friends);
});
}

```

src/navigation/RootNavigator.tsx

```

importReactfrom"react";
import {NavigationContainer }from"@react-navigation/native";
import {createNativeStackNavigator }from"@react-navigation/native-stack";
import {ActivityIndicator,View }from"react-native";
import {RootStackParamList }from"../types";
import {useAuth }from"../contexts/AuthContext";
importLoginScreenfrom"../screens/LoginScreen";

```

```

import RegisterScreen from "../screens/RegisterScreen";
import HomeScreen from "../screens/HomeScreen";
import AlarmFormScreen from "../screens/AlarmFormScreen";
import FriendsScreen from "../screens/FriendsScreen";
import FriendAlarmsScreen from "../screens/FriendAlarmsScreen";

const Stack = createNativeStackNavigator<RootStackParamList>();

export default function RootNavigator() {
  const { user, loading } = useAuth();

  if (loading) {
    return (
      <View
        style={{
          flex: 1,
          alignItems: "center",
          justifyContent: "center",
        }}
      >
        <ActivityIndicator size="large" />
      </View>
    );
  }

  return (
    <NavigationContainer>
      <Stack.Navigator>
        {!user ? (
          <>
            <Stack.Screen
              name="Login"
              component={LoginScreen}
              options={{ title: "로그인" }}
            />
            <Stack.Screen
              name="Register"
              component={RegisterScreen}
              options={{ title: "회원가입" }}
            />
          </>
        ) : (
          <>
            <Stack.Screen
              name="Home"
              component={HomeScreen}
              options={{ title: "내 알람" }}
            />
            <Stack.Screen
              name="AlarmForm"
              component={AlarmFormScreen}
              options={{ title: "알람 설정" }}
            />
            <Stack.Screen
              name="Friends"
              component={FriendsScreen}

```

```

options={{ title:"친구 관리" }}
/>
<Stack.Screen
name="FriendAlarms"
component={FriendAlarmsScreen}
options={{ title:"친구 알람" }}
/>
</>
    )}
</Stack.Navigator>
</NavigationContainer>
);
}

```

App.tsx

```

import React, {useEffect }from"react";
import {AuthProvider }from"./src/contexts/AuthContext";
import RootNavigatorfrom"./src/navigation/RootNavigator";
import {requestNotificationPermission }from"./src/services/notificationService";

exportdefaultfunctionApp() {
useEffect(() => {
requestNotificationPermission();
}, []);

return (
<AuthProvider>
<RootNavigator/>
</AuthProvider>
);
}

```

src/components/AlarmItem.tsx

```

import Reactfrom"react";
import {View,Text,TouchableOpacity,StyleSheet }from"react-native";
import {Alarm }from"../types";

typeProps= {
alarm:Alarm;
onToggle: () =>void;
onEdit: () =>void;
onDelete: () =>void;
};

constdayLabels:Record<number,string>= {
1:"일",

```

```

2: "월",
3: "화",
4: "수",
5: "목",
6: "금",
7: "토",
};

export default function AlarmItem({
  alarm,
  onToggle,
  onEdit,
  onDelete,
}: Props) {
  const time = `${String(alarm.hour).padStart(2, "0")}:${String(
    alarm.minute
  ).padStart(2, "0")}`;

  const repeatText =
    alarm.repeatDays.length === 0
    ? "반복 없음"
    : alarm.repeatDays.map((d) => dayLabels[d]).join(", ");

  return (
    <View style={styles.container}>
      <TouchableOpacity onPress={onToggle} style={styles.main}>
        <Text style={styles.time}>{time}</Text>
        <Text style={styles.title}>{alarm.title}</Text>
        <Text style={styles.repeat}>{repeatText}</Text>
        <Text style={alarm.enabled ? styles.enabled : styles.disabled}>
          {alarm.enabled ? "활성화" : "비활성화"}
        </Text>
      </TouchableOpacity>

      <View style={styles.buttons}>
        <TouchableOpacity onPress={onEdit} style={styles.editButton}>
          <Text style={styles.buttonText}>수정</Text>
        </TouchableOpacity>

        <TouchableOpacity onPress={onDelete} style={styles.deleteButton}>
          <Text style={styles.buttonText}>삭제</Text>
        </TouchableOpacity>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    padding: 16,
    backgroundColor: "#fff",
    marginBottom: 12,
    borderRadius: 12,
    borderWidth: 1,
    borderColor: "#ddd",
  },

```

```

main: {
  marginBottom:12,
},
time: {
  fontSize:32,
  fontWeight:"bold",
},
title: {
  fontSize:16,
  marginTop:4,
},
repeat: {
  color:"#666",
  marginTop:4,
},
enabled: {
  marginTop:4,
  color:"green",
  fontWeight:"bold",
},
disabled: {
  marginTop:4,
  color:"gray",
  fontWeight:"bold",
},
},
buttons: {
  flexDirection:"row",
  gap:8,
},
editButton: {
  flex:1,
  backgroundColor:"#222",
  padding:10,
  borderRadius:8,
  alignItems:"center",
},
deleteButton: {
  flex:1,
  backgroundColor:"#d33",
  padding:10,
  borderRadius:8,
  alignItems:"center",
},
buttonText: {
  color:"#fff",
  fontWeight:"bold",
},
});

```

src/screens/LoginScreen.tsx

```

import React, {useState }from"react";
import {
  View,
  TextInput,
  TouchableOpacity,
  Text,
  Alert,
  StyleSheet,
}from"react-native";
import {NativeStackScreenProps }from"@react-navigation/native-stack";
import {RootStackParamList }from"../types";
import {useAuth }from"../contexts/AuthContext";

typeProps=NativeStackScreenProps<RootStackParamList,"Login">;

exportdefaultfunctionLoginScreen({ navigation }:Props) {
  const { login }=useAuth();
  const [email,setEmail]=useState("");
  const [password,setPassword]=useState("");

  constonLogin=async () => {
    try {
      awaitlogin(email,password);
    }catch (error: any) {
      Alert.alert("로그인 실패",error.message);
    }
  };

  return (
    <Viewstyle={styles.container}>
      <Textstyle={styles.title}>공유 알람</Text>

      <TextInput
        style={styles.input}
        placeholder="이메일"
        autoCapitalize="none"
        value={email}
        onChangeText={setEmail}
      />

      <TextInput
        style={styles.input}
        placeholder="비밀번호"
        secureTextEntry
        value={password}
        onChangeText={setPassword}
      />

      <TouchableOpacitystyle={styles.primaryButton}onPress={onLogin}>
        <Textstyle={styles.primaryText}>로그인</Text>
      </TouchableOpacity>

      <TouchableOpacityonPress={() =>navigation.navigate("Register")}>
        <Textstyle={styles.link}>회원가입</Text>
      </TouchableOpacity>
    </View>
  );
}

```

```

</View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 24,
    justifyContent: "center",
    backgroundColor: "#f7f7f7",
  },
  title: {
    fontSize: 32,
    fontWeight: "bold",
    marginBottom: 32,
    textAlign: "center",
  },
  input: {
    backgroundColor: "fff",
    borderWidth: 1,
    borderColor: "ddd",
    borderRadius: 10,
    padding: 14,
    marginBottom: 12,
  },
  primaryButton: {
    backgroundColor: "#111",
    padding: 16,
    borderRadius: 10,
    alignItems: "center",
    marginTop: 8,
  },
  primaryText: {
    color: "fff",
    fontWeight: "bold",
  },
  link: {
    marginTop: 20,
    textAlign: "center",
    color: "#333",
    fontWeight: "bold",
  },
});

```

src/screens/RegisterScreen.tsx

```

import React, {useState } from "react";
import {
  View,
  TextInput,
  TouchableOpacity,
  Text,

```

```

Alert,
StyleSheet,
}from"react-native";
import {NativeStackScreenProps }from"@react-navigation/native-stack";
import {RootStackParamList }from"../types";
import {useAuth }from"../contexts/AuthContext";

typeProps=NativeStackScreenProps<RootStackParamList,"Register">;

exportdefaultfunctionRegisterScreen({ navigation }:Props) {
  const { register }=useAuth();
  const [email,setEmail]=useState("");
  const [password,setPassword]=useState("");

  constonRegister=async () => {
    try {
      awaitregister(email,password);
    }catch (error: any) {
      Alert.alert("회원가입 실패",error.message);
    }
  };

  return (
    <Viewstyle={styles.container}>
    <Textstyle={styles.title}>회원가입</Text>

    <TextInput
      style={styles.input}
      placeholder="이메일"
      autoCapitalize="none"
      value={email}
      onChangeText={setEmail}
    />

    <TextInput
      style={styles.input}
      placeholder="비밀번호"
      secureTextEntry
      value={password}
      onChangeText={setPassword}
    />

    <TouchableOpacitystyle={styles.primaryButton}onPress={onRegister}>
    <Textstyle={styles.primaryText}>가입하기</Text>
    </TouchableOpacity>

    <TouchableOpacityonPress={() =>navigation.goBack()}>
    <Textstyle={styles.link}>로그인으로 돌아가기</Text>
    </TouchableOpacity>
  </View>
  );
}

conststyles=StyleSheet.create({
  container: {
    flex:1,

```

```

padding:24,
justifyContent:"center",
backgroundColor:"#f7f7f7",
},
title: {
  fontSize:32,
  fontWeight:"bold",
  marginBottom:32,
  textAlign:"center",
},
input: {
  backgroundColor:"#fff",
  borderWidth:1,
  borderColor:"#ddd",
  borderRadius:10,
  padding:14,
  marginBottom:12,
},
primaryButton: {
  backgroundColor:"#111",
  padding:16,
  borderRadius:10,
  alignItems:"center",
  marginTop:8,
},
primaryText: {
  color:"#fff",
  fontWeight:"bold",
},
link: {
  marginTop:20,
  textAlign:"center",
  color:"#333",
  fontWeight:"bold",
},
});

```

src/screens/HomeScreen.tsx

```

import React, {useEffect,useState }from"react";
import {
  View,
  Text,
  FlatList,
  TouchableOpacity,
  Alert,
  StyleSheet,
}from"react-native";
import {NativeStackScreenProps }from"@react-navigation/native-stack";
import {RootStackParamList,Alarm }from"../types";
import {useAuth }from"../contexts/AuthContext";
import {

```

```

deleteAlarm,
subscribeAlarms,
toggleAlarm,
}from"../services/alarmService";
importAlarmItemfrom"../components/AlarmItem";

typeProps=NativeStackScreenProps<RootStackParamList,"Home">;

exportdefaultfunctionHomeScreen({ navigation }:Props) {
  const { user, logout }=useAuth();
  const [alarms,setAlarms]=useState<Alarm[]>([]);

  useEffect(() => {
    if (!user)return;

    constunsub=subscribeAlarms(user.uid,setAlarms);
    returnunsub;
    }, [user]);

    if (!user)returnnull;

    constonDelete= (alarm:Alarm) => {
      Alert.alert("알람 삭제","정말 삭제할까요?", [
        { text:"취소", style:"cancel" },
        {
          text:"삭제",
          style:"destructive",
          onPress:async () => {
            awaitdeleteAlarm(user.uid,user.uid,alarm);
          },
        },
      ]);
    };

    return (
      <Viewstyle={styles.container}>
      <Viewstyle={styles.top}>
      <TouchableOpacity
        style={styles.smallButton}
        onPress={() =>navigation.navigate("Friends")}
      >
      <Textstyle={styles.smallButtonText}>친구</Text>
      </TouchableOpacity>

      <TouchableOpacitystyle={styles.smallButton}onPress={logout}>
      <Textstyle={styles.smallButtonText}>로그아웃</Text>
      </TouchableOpacity>
      </View>

      <TouchableOpacity
        style={styles.addButton}
        onPress={() =>
          navigation.navigate("AlarmForm", {
            ownerId:user.uid,
          })
        }
      >

```

```

>
<Textstyle={styles.addButtonText}>+ 내 알람 추가</Text>
</TouchableOpacity>

<FlatList
  data={alarms}
  keyExtractor={(item) =>item.id}
  ListEmptyComponent={
    <Textstyle={styles.empty}>등록된 알람이 없습니다.</Text>
  }
  renderItem={({ item }) => (
    <AlarmItem
      alarm={item}
      onToggle={() =>toggleAlarm(user.uid,user.uid,item)}
      onEdit={() =>
        navigation.navigate("AlarmForm", {
          ownerId:user.uid,
          alarmId:item.id,
        })
      }
      onDelete={() =>onDelete(item)}
    />
  )}
/>
</View>
);
}

conststyles=StyleSheet.create({
  container: {
    flex:1,
    padding:16,
    backgroundColor:"#f7f7f7",
  },
  top: {
    flexDirection:"row",
    justifyContent:"space-between",
    marginBottom:12,
  },
  smallButton: {
    backgroundColor:"#333",
    paddingVertical:10,
    paddingHorizontal:14,
    borderRadius:8,
  },
  smallButtonText: {
    color:"#fff",
    fontWeight:"bold",
  },
  addButton: {
    backgroundColor:"#111",
    padding:16,
    borderRadius:12,
    alignItems:"center",
    marginBottom:16,
  },
},

```

```

    addButtonText: {
      color: "#fff",
      fontWeight: "bold",
    },
    empty: {
      textAlign: "center",
      marginTop: 40,
      color: "#777",
    },
  },
});

```

src/screens/AlarmFormScreen.tsx

```

import React, {useEffect,useState }from"react";
import {
  View,
  Text,
  TextInput,
  TouchableOpacity,
  Alert,
  StyleSheet,
  ScrollView,
}from"react-native";
import {NativeStackScreenProps }from"@react-navigation/native-stack";
import {RootStackParamList }from"../types";
import {useAuth }from"../contexts/AuthContext";
import {createAlarm,getAlarm,updateAlarm }from"../services/alarmService";

typeProps=NativeStackScreenProps<RootStackParamList,"AlarmForm">;

constdays= [
  { label:"일", value:1 },
  { label:"월", value:2 },
  { label:"화", value:3 },
  { label:"수", value:4 },
  { label:"목", value:5 },
  { label:"금", value:6 },
  { label:"토", value:7 },
];

exportdefaultfunctionAlarmFormScreen({ route, navigation }:Props) {
  const { user }=useAuth();
  const { ownerId, alarmId }=route.params;

  const [title,setTitle]=useState("알람");
  const [hour,setHour]=useState("7");
  const [minute,setMinute]=useState("0");
  const [repeatDays,setRepeatDays]=useState<number[]>([]);
  const [enabled,setEnabled]=useState(true);

  useEffect(() => {
    asyncfunctionload() {

```

```

if (!alarmId) return;

const alarm = await getAlarm(ownerId, alarmId);

if (!alarm) return;

setTitle(alarm.title);
setHour(String(alarm.hour));
setMinute(String(alarm.minute));
setRepeatDays(alarm.repeatDays);
setEnabled(alarm.enabled);
}

load();
}, [ownerId, alarmId]);

if (!user) return null;

const toggleDay = (day: number) => {
  setRepeatDays((prev) =>
    prev.includes(day) ? prev.filter((d) => d !== day) : [...prev, day]
  );
};

const onSave = async () => {
  const h = Number(hour);
  const m = Number(minute);

  if (!title.trim()) {
    Alert.alert("확인", "알람 제목을 입력하세요.");
    return;
  }

  if (Number.isNaN(h) || h < 0 || h > 23) {
    Alert.alert("확인", "시간은 0부터 23 사이로 입력하세요.");
    return;
  }

  if (Number.isNaN(m) || m < 0 || m > 59) {
    Alert.alert("확인", "분은 0부터 59 사이로 입력하세요.");
    return;
  }

  const payload = {
    title: title.trim(),
    hour: h,
    minute: m,
    repeatDays: repeatDays.sort(),
    enabled,
  };

  try {
    if (alarmId) {
      await updateAlarm(ownerId, user.uid, alarmId, payload);
    } else {
      await createAlarm(ownerId, user.uid, payload);
    }
  }

```

```

    }

    navigation.goBack();
  } catch (error: any) {
    Alert.alert("저장 실패", error.message);
  }
};

return (
  <ScrollView contentContainerStyle={styles.container}>
    <Text style={styles.label}>알람 제목</Text>
    <TextInput style={styles.input} value={title} onChangeText={setTitle}/>

    <Text style={styles.label}>시간</Text>
    <View style={styles.row}>
      <TextInput
        style={[styles.input, styles.timeInput]}
        keyboardType="number-pad"
        value={hour}
        onChangeText={setHour}
      />
      <Text style={styles.colon}>:</Text>
      <TextInput
        style={[styles.input, styles.timeInput]}
        keyboardType="number-pad"
        value={minute}
        onChangeText={setMinute}
      />
    </View>

    <Text style={styles.label}>반복 요일</Text>
    <View style={styles.dayRow}>
      {days.map((day) => {
        const selected = repeatDays.includes(day.value);

        return (
          <TouchableOpacity
            key={day.value}
            onPress={() => toggleDay(day.value)}
            style={[styles.dayButton, selected && styles.dayButtonSelected]}
          >
            <Text
              style={[
                styles.dayButtonText,
                selected && styles.dayButtonSelectedText,
              ]}
            >
              {day.label}
            </Text>
          </TouchableOpacity>
        );
      })}
    </View>

    <TouchableOpacity
      style={[styles.statusButton, enabled ? styles.enabled : styles.disabled]}
    >

```

```

onPress={() =>setEnabled((prev) =>!prev)}
>
<Textstyle={styles.statusText}>
    {enabled?"활성화 상태":"비활성화 상태"}
</Text>
</TouchableOpacity>

<TouchableOpacitystyle={styles.saveButton}onPress={onSave}>
<Textstyle={styles.saveButtonText}>저장</Text>
</TouchableOpacity>
</ScrollView>
);
}

```

```

const styles=StyleSheet.create({
  container: {
    padding:20,
    backgroundColor:"#f7f7f7",
    flexGrow:1,
  },
  label: {
    fontWeight:"bold",
    marginBottom:8,
    marginTop:16,
  },
  input: {
    backgroundColor:"#fff",
    borderWidth:1,
    borderColor:"#ddd",
    borderRadius:10,
    padding:14,
  },
  row: {
    flexDirection:"row",
    alignItems:"center",
  },
  timeInput: {
    flex:1,
    textAlign:"center",
    fontSize:24,
  },
  colon: {
    fontSize:32,
    marginHorizontal:10,
  },
  dayRow: {
    flexDirection:"row",
    justifyContent:"space-between",
  },
  dayButton: {
    width:42,
    height:42,
    borderRadius:21,
    borderWidth:1,
    borderColor:"#aaa",
    alignItems:"center",

```

```

        justifyContent:"center",
        backgroundColor:"#fff",
    },
    dayButtonSelected: {
        backgroundColor:"#111",
    },
    },
    dayButtonText: {
        color:"#111",
        fontWeight:"bold",
    },
    },
    dayButtonSelectedText: {
        color:"#fff",
    },
    },
    statusButton: {
        marginTop:24,
        padding:16,
        borderRadius:10,
        alignItems:"center",
    },
    },
    enabled: {
        backgroundColor:"green",
    },
    },
    disabled: {
        backgroundColor:"gray",
    },
    },
    statusText: {
        color:"#fff",
        fontWeight:"bold",
    },
    },
    saveButton: {
        marginTop:24,
        backgroundColor:"#111",
        padding:16,
        borderRadius:12,
        alignItems:"center",
    },
    },
    saveButtonText: {
        color:"#fff",
        fontWeight:"bold",
    },
    },
});

```

src/screens/FriendsScreen.tsx

```

import React, {useEffect,useState }from"react";
import {
    View,
    Text,
    TextInput,
    TouchableOpacity,
    FlatList,
    Alert,

```

```

StyleSheet,
}from"react-native";
import {NativeStackScreenProps }from"@react-navigation/native-stack";
import {RootStackParamList,Friend,FriendRequest }from"../types";
import {useAuth }from"../contexts/AuthContext";
import {
acceptFriendRequest,
sendFriendRequest,
subscribeFriends,
subscribeIncomingRequests,
}from"../services/friendService";

typeProps=NativeStackScreenProps<RootStackParamList,"Friends">;

exportdefaultfunctionFriendsScreen({ navigation }:Props) {
const { user }=useAuth();
const [targetEmail,setTargetEmail]=useState("");
const [friends,setFriends]=useState<Friend[]>([]);
const [requests,setRequests]=useState<FriendRequest[]>([]);

useEffect(() => {
if (!user)return;

constunsubFriends=subscribeFriends(user.uid,setFriends);
constunsubRequests=subscribeIncomingRequests(user.uid,setRequests);

return () => {
unsubFriends();
unsubRequests();
};
}, [user]);

if (!user)returnnull;

constonSendRequest=async () => {
try {
awaitsendFriendRequest(
user.uid,
user.email??"",
targetEmail.trim()
);

setTargetEmail("");
Alert.alert("완료","친구 요청을 보냈습니다.");
}catch (error: any) {
Alert.alert("요청 실패",error.message);
}
};

return (
<Viewstyle={styles.container}>
<Textstyle={styles.sectionTitle}>친구 추가</Text>

<Viewstyle={styles.inputRow}>
<TextInput
style={styles.input}

```

```

placeholder="친구 이메일"
autoCapitalize="none"
value={targetEmail}
onChangeText={setTargetEmail}
/>

<TouchableOpacity style={styles.sendButton} onPress={onSendRequest}>
<Text style={styles.sendButtonText}>요청</Text>
</TouchableOpacity>
</View>

<Text style={styles.sectionTitle}>받은 친구 요청</Text>

<FlatList
data={requests}
keyExtractor={({ item }) => item.id}
ListEmptyComponent={<Text style={styles.empty}>받은 요청 없음</Text>}
renderItem={({ item }) => (
<View style={styles.card}>
<Text>{item.fromEmail}</Text>
<TouchableOpacity
style={styles.acceptButton}
onPress={() => acceptFriendRequest(item)}
>
<Text style={styles.buttonText}>수락</Text>
</TouchableOpacity>
</View>
)}
/>

<Text style={styles.sectionTitle}>친구 목록</Text>

<FlatList
data={friends}
keyExtractor={({ item }) => item.uid}
ListEmptyComponent={<Text style={styles.empty}>친구 없음</Text>}
renderItem={({ item }) => (
<TouchableOpacity
style={styles.card}
onPress={() =>
navigation.navigate("FriendAlarms", {
friendId: item.uid,
friendEmail: item.email,
})
}
>
<Text style={styles.friendEmail}>{item.email}</Text>
<Text style={styles.hint}>알람 관리하기</Text>
</TouchableOpacity>
)}
/>
</View>
);
}

const styles = StyleSheet.create({

```

```

container: {
  flex:1,
  padding:16,
  backgroundColor:"#f7f7f7",
},
sectionTitle: {
  fontSize:18,
  fontWeight:"bold",
  marginTop:16,
  marginBottom:8,
},
inputRow: {
  flexDirection:"row",
  gap:8,
},
input: {
  flex:1,
  backgroundColor:"fff",
  borderWidth:1,
  borderColor:"ddd",
  borderRadius:10,
  padding:12,
},
sendButton: {
  backgroundColor:"#111",
  paddingHorizontal:16,
  borderRadius:10,
  justifyContent:"center",
},
sendButtonText: {
  color:"fff",
  fontWeight:"bold",
},
card: {
  backgroundColor:"fff",
  padding:14,
  borderRadius:10,
  marginBottom:8,
  borderWidth:1,
  borderColor:"ddd",
},
acceptButton: {
  backgroundColor:"green",
  padding:10,
  borderRadius:8,
  alignItems:"center",
  marginTop:8,
},
buttonText: {
  color:"fff",
  fontWeight:"bold",
},
friendEmail: {
  fontWeight:"bold",
},
hint: {

```

```

        color:"#666",
        marginTop:4,
    },
    empty: {
        color:"#777",
        marginBottom:8,
    },
});

```

src/screens/FriendAlarmsScreen.tsx

```

import React, {useEffect,useState }from"react";
import {
  View,
  Text,
  FlatList,
  TouchableOpacity,
  Alert,
  StyleSheet,
}from"react-native";
import {NativeStackScreenProps }from"@react-navigation/native-stack";
import {RootStackParamList,Alarm }from"../types";
import {useAuth }from"../contexts/AuthContext";
import {
  deleteAlarm,
  subscribeAlarms,
  toggleAlarm,
}from"../services/alarmService";
import AlarmItemfrom"../components/AlarmItem";

typeProps=NativeStackScreenProps<RootStackParamList,"FriendAlarms">;

exportdefaultfunctionFriendAlarmsScreen({ route, navigation }:Props) {
  const { user }=useAuth();
  const { friendId, friendEmail }=route.params;
  const [alarms,setAlarms]=useState<Alarm[]>([]);

  useEffect(() => {
    constunsub=subscribeAlarms(friendId,setAlarms);
    returnunsub;
  }, [friendId]);

  if (!user)returnnull;

  constonDelete=( alarm:Alarm) => {
    Alert.alert("알람 삭제","친구의 알람을 삭제할까요?", [
      { text:"취소", style:"cancel" },
      {
        text:"삭제",
        style:"destructive",
        onPress:async () => {
          awaitdeleteAlarm(friendId,user.uid,alarm);

```

```

        },
      },
    ]);
  };

  return (
    <Viewstyle={styles.container}>
      <Textstyle={styles.email}>{friendEmail}</Text>

      <TouchableOpacity
        style={styles.addButton}
        onPress={() =>
          navigation.navigate("AlarmForm", {
            ownerId: friendId,
          })
        }
      >
      <Textstyle={styles.addButtonText}>+ 친구 알람 추가</Text>
    </TouchableOpacity>

    <FlatList
      data={alarms}
      keyExtractor={(item) => item.id}
      ListEmptyComponent={
        <Textstyle={styles.empty}>등록된 알람이 없습니다.</Text>
      }
      renderItem={({ item }) => (
        <AlarmItem
          alarm={item}
          onToggle={() => toggleAlarm(friendId, user.uid, item)}
          onEdit={() =>
            navigation.navigate("AlarmForm", {
              ownerId: friendId,
              alarmId: item.id,
            })
          }
          onDelete={() => onDelete(item)}
        />
      )}
    />
  </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 16,
    backgroundColor: "#f7f7f7",
  },
  email: {
    fontSize: 16,
    fontWeight: "bold",
    marginBottom: 12,
  },
  addButton: {

```

```

        backgroundColor:"#111",
        padding:16,
        borderRadius:12,
        alignItems:"center",
        marginBottom:16,
    },
    addButtonText: {
        color:"#fff",
        fontWeight:"bold",
    },
    empty: {
        textAlign:"center",
        marginTop:40,
        color:"#777",
    },
    });

```

5. Firestore 보안 규칙

firestore.rules

```

rules_version='2';

service cloud.firestore {
  match /databases/{database}/documents {
    function signedIn() {
      return request.auth != null;
    }

    function isMe(uid) {
      return signedIn() && request.auth.uid == uid;
    }

    function isFriend(uid) {
      return signedIn()
      && exists(/databases/{database}/documents/users/{uid}/friends/{request.auth.uid});
    }

    function canAccessUser(uid) {
      return isMe(uid) || isFriend(uid);
    }

    match /users/{uid} {
      allow read: if signedIn();
      allow create: if isMe(uid);
      allow update, delete: if isMe(uid);

      match /friends/{friendUid} {
        allow read: if isMe(uid);
        allow create, update, delete: if isMe(uid);
      }
    }
  }
}

```

```

    }

    match/alarms/{alarmId} {
      allowread:ifcanAccessUser(uid);
      allowcreate:ifcanAccessUser(uid);
      allowupdate:ifcanAccessUser(uid);
      allowdelete:ifcanAccessUser(uid);
    }
  }

  match/friendRequests/{requestId} {
    allowread:ifsignedIn()
    && (
      resource.data.fromUid==request.auth.uid
      ||resource.data.toUid==request.auth.uid
    );

    allowcreate:ifsignedIn()
    &&request.resource.data.fromUid==request.auth.uid
    &&request.resource.data.status=="pending";

    allowupdate:ifsignedIn()
    &&resource.data.toUid==request.auth.uid;

    allowdelete:iffalse;
  }
}

```

6. 실행

```
npx expo start
```

Android:

```
npx expo run:android
```

iOS:

```
npx expo run:ios
```

7. 중요한 제한 사항

현재 코드는 **MVP로 바로 실행 가능한 구조**입니다. 다만 실제 배포 수준으로 가려면 아래 보완이 필요합니다.

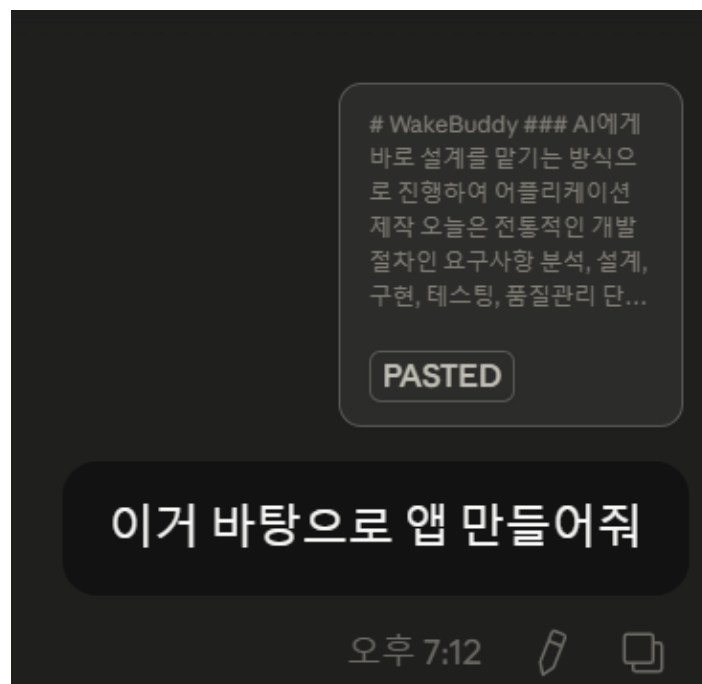
친구가 내 알람을 추가했을 때, 그 알람이 내 기기에서 울리려면 **내 앱이 Firestore 변경 사항을 받아서 로컬 알람으로 등록해야 합니다**. 위 코드는 내 알람을 내가 만들거나 수정할 때는 즉시 로컬 알람이 등록되지만, 친구가 원격으로 추가한 알람을 내 기기에서 항상 백그라운드로 자동 등록하는 것은 모바일 OS 제약 때문에 완벽하지 않습니다.

실제 서비스 수준으로 만들려면 다음 중 하나가 필요합니다.

1. Firebase Cloud Functions + FCM Push Notification
2. 앱 실행 시 Firestore 알람 동기화 후 로컬 알람 재등록
3. Android 네이티브 AlarmManager / iOS UNUserNotificationCenter 직접 연동
4. Expo Managed가 아닌 Bare Workflow 또는 React Native CLI 전환

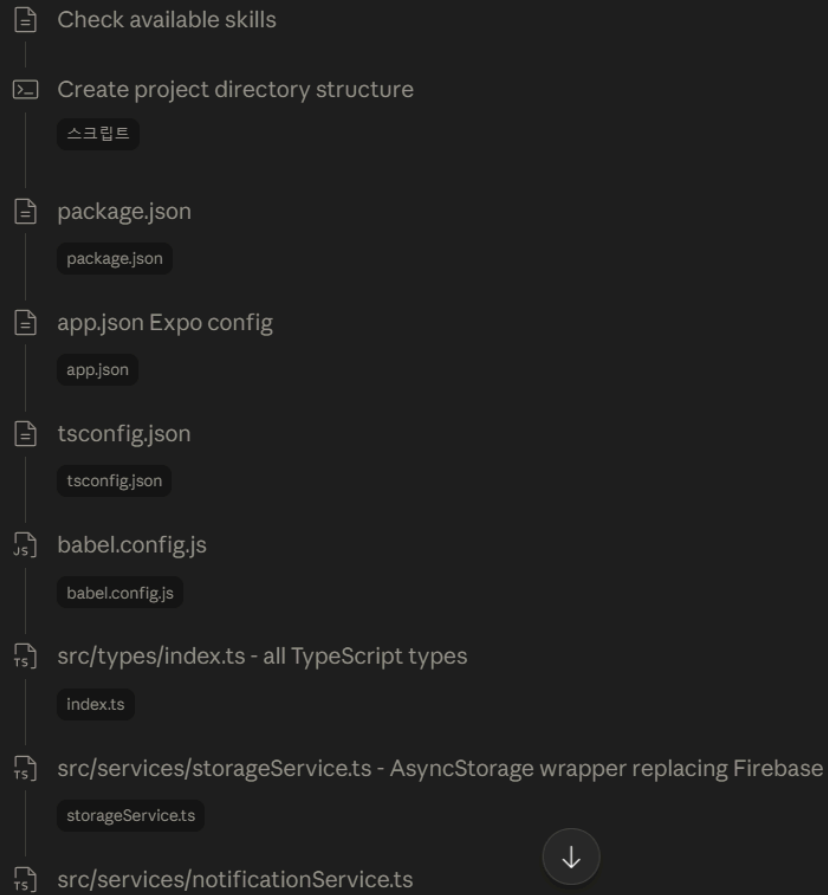
즉, **수업 프로젝트 / MVP / 시연용**이면 위 코드로 충분하고, **실제 앱스토어 배포용**이면 알람 백그라운드 안정성을 위해 네이티브 모듈 보강이 필요합니다.

Claude 에 돌린 결과



알겠습니다! React Native + Expo 프로젝트를 전체를 zip으로 묶어드릴게요. Firebase 대신 Mock 데이터와 AsyncStorage 기반으로 로컬에서 완전히 동작하도록 구현합니다.

20개의 파일 생성됨, 파일 조회함, 명령 실행함 ▾



AlarmItem.tsx

src/screens/LoginScreen.tsx

LoginScreen.tsx

src/screens/RegisterScreen.tsx

RegisterScreen.tsx

src/screens/HomeScreen.tsx

HomeScreen.tsx

src/screens/AlarmFormScreen.tsx

AlarmFormScreen.tsx

src/screens/FriendsScreen.tsx

FriendsScreen.tsx

src/screens/FriendAlarmsScreen.tsx

FriendAlarmsScreen.tsx

App.tsx - root entry point

App.tsx

완료

authService에 import 충돌이 있어서 수정합니다.

Fix authService to remove circular import >

이제 README와 zip을 만들겠습니다.

명령 2개 실행함, 파일 생성됨, 파일 읽음 >

완성됐습니다! 압축 파일에 아래 내용이 전부 포함되어 있어요.



Wakebuddy final
ZIP

다운로드

claude가 구현한 화면들

